

Цели практической работы:

Научиться:

- создавать свои сервисы на Laravel;
- работать с логами Laravel и их обработкой.

Что нужно сделать:

В этой практической работе вы разработаете сервис логирования, который:

- фиксирует обращения к сайту;
- собирает их в базе данных с возможностью отключения системы логирования;
- отражает в реальном времени HTTP-запросы к приложению.

Создадим новый проект:

```
composer create-project laravel/laravel log-service
```

1. Для начала создадим модель логов. Для создания модели необходимо использовать **artisan** с параметром **make:model**.

В итоге наша команда будет выглядеть так:

```
php artisan make:model Log
```

По умолчанию модель создаётся в `./app/Models/Log.php`.

Модель создана, для избежания ошибок запросов SQL необходимо отключить автоматические метки времени.

```
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Model;
6
7  class Log extends Model
8  {
9      public $timestamps = false;
10 }
```

2. Теперь опишем миграцию для создания нашей таблицы логов:

```
php artisan make:migration create_logs_table
```

Напомним, что таблицы миграции создаются по умолчанию в `/database/migration/current_date_time_create_logs_table.php`.

По умолчанию создаётся файл, содержимое которого выглядит так:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('logs', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('logs');
    }
};
```

В этом файле нам нужно определить поля, которые будет собирать наш сервис логирования:

- time — время события;
- duration — длительность;
- IP — IP-адрес зашедшего пользователя;
- url — адрес, который запросил пользователь;
- method — HTTP-метод (GET, POST);
- input — передаваемые параметры.

В итоге файл должен приобрести такой вид:

```

<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('logs', function (Blueprint $table) {
            $table->bigIncrements('id');
            $table->dateTime('time');
            $table->integer('duration');
            $table->string('ip',100)->nullable();
            $table->string('url')->nullable();
            $table->string('method',10)->nullable();
            $table->string('input')->nullable();
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('logs');
    }
};

```

3. Миграция создана, параметры описаны. Теперь создадим таблицу.

Напоминаем, что таблица создаётся также через **artisan** с параметром **migrate php artisan migrate**.

4. База данных подготовлена, теперь нужно создать звено (middleware) для обработки HTTP-запросов. Напоминаем, что звенья создаются при помощи команды **php artisan make:middleware название модели**.

В нашем случае нам нужна команда:
php artisan make:middleware DataLogger

По умолчанию звено (посредник) создастся по пути
./app/Http/Middleware/DataLogger.php.

Теперь необходимо настроить middleware. Открываем **Datalogger.php**.
 Добавим использование созданной модели.

```

1 <?php
2
3 namespace App\Http\Middleware;
4
5 use Closure;
6 use App\Models\Log;
7 class DataLogger
8 {
9     private $start_time;
10    /**
11     * Handle an incoming request.
12     *
13     * @param \Illuminate\Http\Request $request
14     * @param \Closure(\Illuminate\Http\Request): (\Illuminate\Http\Response|\Illuminate\Http\RedirectResponse) $next
15     * @return \Illuminate\Http\Response|\Illuminate\Http\RedirectResponse
16     */
17    public function handle($request, Closure $next)
18    {
19        $this->start_time = microtime(true);
20        return $next($request);
21    }
22    public function terminate($request, $response) //Функция, которая вызывается после получения ответа пользователем
23    {
24        if ( env('API_DATA_LOGGER', true) ) { //Если в env файле прописана опция API_DATA_LOGGER = true используем логирование
25            if ( env('API_DATA_LOGGER_USE_DB', true) ) { //Если в env файле прописана опция API_DATA_LOGGER_USE_DB=true,
26                //то для сохранения используем БД иначе пишем просто в файл
27                $endTime = microtime(true);
28                $log = new Log();
29                $log->time = gmdate('Y-m-d H:i:s');
30                $log->duration = number_format($endTime - LARAVEL_START, 3);
31                $log->ip = $request->ip();
32                $log->url = $request->fullUrl();
33                $log->method = $request->method();
34                $log->input = $request->getContent();
35                $log->save(); //Сохраняем в Базу нашу запись
36            }
37        } else //На всякий случай, если опция записи в БД недоступна пишем в файл
38        { //На всякий случай, если опция записи в БД недоступна пишем в файл
39            $endTime = microtime(true);
40            $filename = 'api_data_logger_' . date('d-m-y') . '.log';
41            $dataToLog = "Time: " . gmdate("F j, Y, g:i a") . "\n";
42            $dataToLog .= "Duration: " . number_format($endTime - LARAVEL_START, 3) . "\n";
43            $dataToLog .= "IP Address: " . $request->ip() . "\n";
44            $dataToLog .= "URL: " . $request->fullUrl() . "\n";
45            $dataToLog .= "Method: " . $request->method() . "\n";
46            $dataToLog .= "Input: " . $request->getContent() . "\n";
47            \File::append( storage_path('logs' . DIRECTORY_SEPARATOR . $filename), $dataToLog . "\n" . str_repeat(" ", 20) . "\n\n"); //Записываем в файл
48        }
49    }
50 }
51 }

```

Также нужно завершить создание middleware DataLogger, зарегистрировать его в ./app/Http/Kernel.php.

```

protected $middleware = [
    // \App\Http\Middleware\TrustHosts::class,
    \App\Http\Middleware\TrustProxies::class,
    \Illuminate\Http\Middleware\HandleCors::class,
    \App\Http\Middleware\PreventRequestsDuringMaintenance::class,
    \Illuminate\Foundation\Http\Middleware\ValidatePostSize::class,
    \App\Http\Middleware\TrimStrings::class,
    \Illuminate\Foundation\Http\Middleware\ConvertEmptyStringsToNull::class,
    \App\Http\Middleware\DataLogger::class,
];

```

5. Модель создана, посредник HTTP-запросов настроен и зарегистрирован как класс в Kernel.php. Если сейчас запустить Laravel командой `php artisan serv`, всё будет работать. Логи будут записываться в базу данных. Но увидеть это можно только в самой базе SQL. Для получения более наглядных результатов необходимо создать в `web.php` эндпоинт.

```

Route::get('/logs',function() {
    return view('logs');
});

```

Также для этого эндпоинта необходимо создать blade-шаблон:

./resource/view/logs.blade.php

В нём создать запрос к базе SQL и вывод логов в таблицу.

```
1 <DOCTYPE html>
2 <html style="background: #2C2B2C;">
3 <head>
4 <meta charset="UTF-8">
5 <title>Logs</title>
6 <style>
7     td:nth-child(5),td:nth-child(6){text-align:center;}
8     table(position: absolute; border-spacing: 0;border-collapse: collapse;width: 70%;box-shadow: 0px 4px 100px rgb(255 255 255 / 25%);
9     td, th{padding: 10px;border: 1px solid #2B2B2B;}
10    tr:nth-child(odd) { background-color: #C1B7B7; }
11 </style>
12 </head>
13 <body>
14 <?php
15 $db_server = "127.0.0.1";
16 $db_user = "root";
17 $db_password = "";
18 $db_name = "laravel";
19
20 try {
21     $db = new PDO("mysql:host=$db_server;dbname=$db_name", $db_user, $db_password,array(PDO::MYSQL_ATTR_INIT_COMMAND=>"SET NAMES utf8"));
22     $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
23
24     $sql = "SELECT id, time, duration, ip, url, method, input FROM logs";
25
26     $statement = $db->prepare($sql);
27
28     $statement->execute();
29     $statement->execute();
30
31     $result_array = $statement->fetchAll();
32
33     echo "<div class='table'>";
34     echo "<table><tr><th>id</th><th>time</th><th>duration</th><th>ip</th><th>url</th><th>method</th><th>input</th></tr>";
35     foreach ($result_array as $result_row) {
36         echo "<tr>";
37         echo "<td align='center'>" . $result_row["id"] . "</td>";
38         echo "<td align='center'>" . $result_row["time"] . "</td>";
39         echo "<td align='center'>" . $result_row["duration"] . "</td>";
40         echo "<td align='center'>" . $result_row["ip"] . "</td>";
41         echo "<td align='center'>" . $result_row["url"] . "</td>";
42         echo "<td align='center'>" . $result_row["method"] . "</td>";
43         echo "<td align='center'>" . $result_row["input"] . "</td>";
44         echo "</tr>";
45     }
46     echo "</table>";
47     echo "</div>";
48 }
49
50 catch(PDOException $e) {
51     echo "Ошибка при создании записи в базе данных: " . $e->getMessage();
52 }
53
54
55 $db = null;
56 ?>
57 </body>
58 </html>
```

Запускаем приложение, при открытии вашего приложения <http://localhost:8000/logs> должна открываться таблица с логами обращения к сайту.

Что оценивается:

Принято:

- Выполнены все основные пункты работы.
- Сервис запускается без ошибок.
- Логи HTTP-запросов фиксируются в БД.
- Логи отображаются в веб-интерфейсе в реальном времени.

На доработку: Работа выполнена не полностью или с ошибками.

Как отправить работу на проверку:

Отправьте коммит, содержащий код задания, на ветку master в вашем репозитории и пришлите его URL (URL Merge Request'a) через форму. Репозиторий должен быть public.